Model Extraction Attacks on Graph Neural Networks: Taxonomy and Realisation

Bang Wu
bang.wu@monash.edu
Monash University
Melbourne, Australia

Xiangwen Yang
wayne.yang@monash.edu
Monash University
Melbourne, Australia

Shirui Pan
shirui.pan@monash.edu
Monash University
Melbourne, Australia

Xingliang Yuan
xingliang.yuan@monash.edu
Monash University
Melbourne, Australia

ABSTRACT

Machine learning models are shown to face a severe threat from Model Extraction Attacks, where a well-trained private model owned by a service provider can be stolen by an attacker pretending as a client. Unfortunately, prior works focus on the models trained over the Euclidean space, e.g., images and texts, while how to extract a GNN model that contains a graph structure and node features is yet to be explored. In this paper, for the first time, we comprehensively investigate and develop model extraction attacks against GNN models. We first systematically formalise the threat modelling in the context of GNN model extraction and classify the adversarial threats into seven categories by considering different background knowledge of the attacker, e.g., attributes and/or neighbour connections of the nodes obtained by the attacker. Then we present detailed methods which utilise the accessible knowledge in each threat to implement the attacks. By evaluating over three real-world datasets, our attacks are shown to extract duplicated models effectively, i.e., 84% - 89% of the inputs in the target domain have the same output predictions as the victim model.

CCS CONCEPTS

• **Security and privacy**; • **Computing methodologies** → **Machine learning**;

KEYWORDS

Graph Neural Networks; Model Extraction Attack

ACM Reference Format:
Bang Wu, Xiangwen Yang, Shirui Pan, and Xingliang Yuan. 2022. Model Extraction Attacks on Graph Neural Networks: Taxonomy and Realisation. In *Proceedings of the 2022 ACM Asia Conference on Computer and Communications Security (ASIA CCS '22), May 30-June 3, 2022, Nagasaki, Japan.* ACM, New York, NY, USA, 14 pages. https://doi.org/10.1145/3488932.3497753

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASIA CCS '22, May 30-June 3, 2022, Nagasaki, Japan
© 2022 Association for Computing Machinery.
ACM ISBN 978-1-4503-9140-5/22/05…$15.00
https://doi.org/10.1145/3488932.3497753

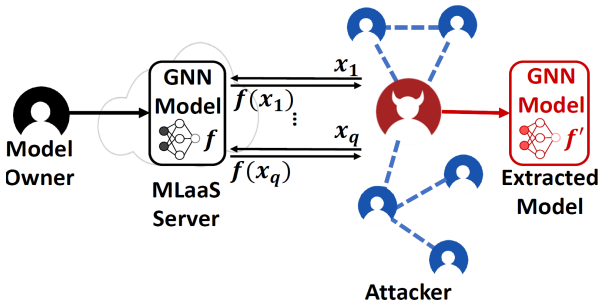


Figure 1: GNNs Model Extraction Attacks. A model owner provides a GNN model f and the service of prediction queries. An attacker extracts a surrogate model $f' \approx f$ based on the answers from the server.

1 INTRODUCTION

Graph data are ubiquitously used in many applications, e.g., social media, document collections, and rating networks [21, 26]. To substantially analyse the graphs, graph neural networks (GNNs), as graph-based machine learning (ML) models, have been increasingly explored and offered state-of-the-art performance [15, 26, 43, 53]. As known, a well-trained machine model is costly during the data gathering, training period, and is often considered as the intellectual property of its owner [54]. To cater for the demands, cloud/AI platforms, e.g., Amazon SageMaker and Google Cloud AutoML, provide privatisation deployments for model owners to sell their models with a licensing fee [40]. Besides, GNN models used for e-commerce recommendation [34] also provide public API to vast customers. On the other hand, such commercialisation draws much attention to the security of the models.

It has been demonstrated that attackers can steal ML models by Model Extraction Attack [35, 41]. Different from the adversarial attacks which aim at deducting the performance of ML systems, they propose to reconstruct a substitute model from the responses of generated queries to the target model. Because the input-output mappings of these queries contain sufficient information of the model prediction tasks, the extracted model can be quite similar to the target model, i.e., achieving comparable accuracy or generating the same prediction results as the target model. However, existing

[1]The code of the paper is released at https://github.com/TrustworthyGNN/MEA-GNN

attacks only target the models with non-graph structures, e.g., MLP and CNN, while few studies focus on graph data. The threats of GNN model extraction are still unclear.

In this paper, we are the first to systematically explore and develop the model extraction attacks against GNNs. Specifically, referring to a private GNN model as the target, an extraction attacker attempts to construct a duplicated model with similar functionality via a sequence of queries from the nodes they obtained (aka *attack nodes*). Figure 1 illustrates an example of how our attack steals a GNN model on a node classification task of a social network. We consider an extraction attacker blending in with normal users in the network. The attacker generates queries to the target model deployed in a machine learning as a service (MLaaS) and obtains the responses via its APIs. The training of the duplicated model then utilises the information extracted from the input-output pairs.

Unlike attacks to other neural networks, the extraction of GNNs requires knowledge in addition to the input-output mappings. An attacker targeting these graph-based classifiers needs to further consider the contributions of graph structures to classification tasks. Different from the models for non-graph structured data, a GNN model predicts labels based on not only the input nodes but also their connections. For example, the type of an image can be inferred by a CNN model individually, but the prediction of a node label will consider all the attributes of the node and other connected nodes. Accordingly, such knowledge about the graph structure should also be taken into consideration during the model extraction.

The above specific requirement poses new challenges in developing model extraction attacks on GNNs. Existing attack strategies cannot be directly applied to GNNs since they leave out of graph structure during extraction. As mentioned, the attacker in GNNs should additionally gather the graph structure, but such knowledge sometimes cannot be obtained in real-world applications. For example, considering an online social network that contains both public and hidden user information, the attacker may have access to the public data, such as friendships (connections between users) but not to the private data, such as personal interests (attributes of users) [10, 17, 19]. Therefore, how to design model extraction attacks with only missing or incomplete knowledge of the target model training graph is non-trivial.

To address the above challenges, we first propose a comprehensive framework for model extracting attacks against GNNs to understand and capture the capabilities of real-world attackers. We formalise the threat modelling by considering the attackers with diverse background knowledge in practice. The knowledge includes three dimensions: the attributes of the attack nodes, the partial graph consisting of the attack nodes, and an auxiliary sub-graph (shadow graph), including its graph structure and attribute information. This sub-graph can be exclusive to the graphs that are used to train the target models but having similar attributes and graph structure. Then, we characterise our attacks into seven different types of model extraction with or without this knowledge, and realise them using adaptive strategies. Specifically, if an attacker knows the graph structure but lacking node attributes knowledge, we design attribute synthesis algorithms to enrich the node attribute set and improve the attacks. If the obtained graph structure is fragmented, we utilise the known attributes to construct a surrogate graph by graph structure generation methods.

The main contributions of our work are summarised as follows:

- To our best knowledge, for the first time, we systematically develop a series of GNN model extraction attacks that can steal a GNN model. The extracted model behaves similarly to the target victim model.
- We propose a framework of threat modelling in the context of GNNs, which formalises and characterises the attacker's knowledge from three dimensions: node attributes, graph structure, and shadow sub-graph.
- We define seven types of model extraction attacks under the above framework and realise them via adaptive attack strategies. We implement each attack by utilising known background knowledge and constructing a surrogate training graph to build a duplicated model.
- We evaluate our attacks over three real-world datasets. The experiments confirm that, our attacks can effectively extract a duplicated model that is similar to the target model. Most of the duplicated models achieve nearly equal accuracy as the target, and more than 85% prediction results from them are the same as the target.

The rest of the paper is organized as follows. Section 2 discusses related studies about the model extracted attacks targeting at ordinary Deep Learning system's privacy, and other adversarial attacks against GNNs. Section 3 introduce the preliminaries of our design. Section 4 propose the goal, knowledge and the taxonomy of the attacks. Section 5 introduces the detailed attack methodologies for each of our attacks. Section 6 shows our experimental results. Section 7 provides the summary and conclusion of our paper.

2 RELATED WORK

Model Extraction Attacks. Model extraction attacks targeting the confidentiality of ML systems have become paramount and have been explored in lots of studies [13, 18, 33, 36–38]. Tramèr *et al.* [41] propose the first model extraction attacks against the linear ML models via Prediction APIs. They reconstruct the model by solving the equations built by the queries, and the labels or confidence values. They also use a path finding approach to attack the decision tree models. Later, more studies consider attacking complex ML models, e.g., Neural Networks. Milli *et al.* [33] provide a gradient-based algorithm that extracts a two-layer ReLU network by carefully choosing the query inputs. Pal *et al.* [37] demonstrate an attack on DNNs for both image and text classification tasks with active learning strategies. Orekondy *et al.* [36] propose the attack by training a "knockoff" model which aims to match or even exceed the accuracy of the target model by generating query-prediction pairs.

Several approaches have also been proposed to defend against model extraction attacks, but they are not suitable for our attacks. Some of them propose to hide or add noise to the output probabilities while maintaining the label outputs [4, 27, 41]. But they are less effective in facing label-based extraction attacks like our design. Others try to monitor each query and differentiate the adversarial ones by analysing the input distribution or the output entropy [23, 24]. However, they do not consider the graph structure and are not optimised for GNN models.

Notation	Explanation
G	An attributed graph of target model training
V	Node set of G
E	Edge set of G
X	Attribute set of V
Y	Label set of the nodes V
$f_\theta(.)$	A node classification model with parameters θ
P	Prediction result set of $f_\theta(v_i)$ for every node $v_i \in V$
G'	A shadow graph with the same domain as G
$V_{\mathcal{A}}$	Attack node set
$V_{\mathcal{A},k-hop}$	k-hop neighbour node set of the attack nodes $V_{\mathcal{A}}$
$E_{\mathcal{A}}$	Connections among the attack nodes $V_{\mathcal{A}}$
$E^*_{\mathcal{A}}$	Synthetic connections among the attack nodes $V_{\mathcal{A}}$
$E_{\mathcal{A},k-hop}$	k-hop neighbour connections of the attack nodes $V_{\mathcal{A}}$
$X_{\mathcal{A}}$	Attributes of the attack nodes $V_{\mathcal{A}}$
$X^*_{\mathcal{A},k-hop}$	Synthetic attributes of the k-hop neighbours
D_i	Degree of the node v_i

Table 1: Notations

Attacks on Graph Neural Networks. Many studies have explored the vulnerability in GNNs. Most of them are adversarial attacks that target the integrity of the GNN systems [29, 44, 47, 48, 55]. Zügner *et al.* [55] propose a scalable greedy approximation scheme to find the perturbation attacking the node classification GNNs. They evaluate both node attribute and graph structure perturbation and compare their effectiveness. Zhang *et al.* [50] present the a transferable attacks against the graph-level GNNs. Wang *et al.* [44] generate the adversarial inputs by adding fake nodes into existing graphs without manipulating the existing connections. Zhang *et al.* [51] propose a collection of data poisoning attack strategies, by manipulating the facts on the target graph. Xu *et al.* [48] and Zhang *et al.* [52] propose the backdoor attacks by poisoning the training graph. Li *et al.* [28] study the attack on the graph learning-based community detection models via hiding a set of nodes based on their surrogate model.

Recent studies also draw attention to attacking the confidentiality of GNNs. A set of advanced attacks called membership inference attacks aim to infer whether a data sample has been used during the target model training [17, 22, 42]. Besides, He *et al.* [20] apply link stealing attacks against GNNs which can infer whether there is a link between two nodes on their training graph. Wu *et al.* [2] propose the membership inference attacks against the graph-level GNN classifiers. Most of them target the components of the graph rather than the GNN models, and our work aims to fill this gap.

3 PRELIMINARIES

In this section, we review a typical task of GNNs, and then proceed with the architecture of the target models, which is prevalently used to evaluate the attacks in GNNs. Our attacks under these typical scenarios can also be extended to the GNN models with other architectures.

Node Classification. Given an attributed graph $G = (V, E, X)$, a set of nodes V with node features X are connected by a set of edges E. A node classification model $f(.)$ can assign node labels Y to each node in V corresponding to both their node features and the graph structure. We denote a classifier with parameters θ as $f_\theta(.)$. The classification result for this model on a node v_i (where $v_i \in V$) in G is designated as $p_i = f_\theta(v_i)$. For a well-trained GNN model, p_i is expected to be the same as y_i, the corresponding label of v_i.

Graph Convolution Networks. In this paper, we consider a general graph convolution network (GCN) [25], indicated by $f_\theta(.)$, for the node classification task. GCN contains convolution layers that aggregate attribute information from the neighbour nodes. The equation for a two-layer GCN is defined as:

$$f_\theta(A, X) = softmax(\hat{A} \cdot ReLU(\hat{A} \cdot X \cdot W^{(0)}) \cdot W^{(1)}), \quad (1)$$

where $\hat{A} = \hat{D}^{-1/2}\widetilde{A}\hat{D}^{-1/2}$ denotes the normalised adjacency matrix, $\widetilde{A} = A + I_N$ denotes adding the identity matrix I_N to the adjacent matrix A. $\hat{D}$ is the diagonal matrix with on-diagonal element as $\hat{D}_{ii} = \sum_j \widetilde{A}_{ij}$. $W^{(0)}$ and $W^{(1)}$ are the weights of first and second layer of GCN, respectively. $ReLU(0, a) = max(0, a)$ is adopted. The notations we use throughout the paper are summarised in Table 1. Considering the model extraction attacks against GCNs, the parameters of the targeted model are W=$\{W^{(0)}, W^{(1)}\}$. As a result, the goal of the attacks targeting a GCN model becomes reconstructing the weights W.

4 ATTACK STATEMENT

4.1 Adversary's Goal

Considering an MLaaS system, a private model provided by an entity can be deployed to the cloud server. This server provides a query interface to the clients, while the clients can issue queries to the server and receive the responses. The model extraction attack aims to utilise the information derived from these input-output query pairs, extract the knowledge about the private model, and reconstruct a surrogate model.

Formally, we consider a GNN model $f_\theta(.)$ trained on an attributed graph $G = (V, E, X)$ for a node classification task. The model extraction attack attempts to reconstruct a surrogate model $f_{\theta'}(.)$ such that $\forall v_i \in V, f'_{\theta'}(v_i) \approx f_\theta(v_i)$, where V is a set of all the nodes in the graph and v_i is one of the nodes.

We define a successful attack as one in which the attacker constructs a model that achieves similar performance to the target model (e.g. achieving similar accuracy at the testing set, or providing similar output predictions). To achieve this objective, the extracted model parameters do not need to be identical to the targeted ones. Namely, the model weights, or even the structure of the model, may be different than the target models once they have the same performance as the target models. The above definition is consistent with the one used in the ordinary DNN system. In practice, it is sufficient to harm the privacy of GNNs if a model with similar performance is extracted.

4.2 Adversarial Knowledge

Attackers with diverse background knowledge can apply the model extraction attacks at different levels. In this paper, we tackle the most challenging adversarial setting: black-box attacks. Following the black-box assumptions in GNN attacks [5, 32], the attacker may obtain the adjacency matrix, attribute matrix, and output of

Attack	X	A	G'	Attack	X	A	G'
——	○	○	○	Attack-3	○	○	●
Attack-0	◐	◐	○	Attack-4	◐	◐	●
Attack-1	◐	○	○	Attack-5	◐	○	●
Attack-2	○	●	○	Attack-6	○	●	●

Table 2: Taxonomy of the proposed threat model. X represents the target dataset's nodes attributes, A represents the target dataset's graph structure, G' represents a shadow graph, and ●/◐/○means the attacker has complete/partial/no knowledge.

the victim model, while the model parameters, labels and output probability are unknown. In practice, it is reasonable that the attacker may only get access to a set of attack nodes, i.e., a subset of the nodes in the entire graph [32]. Namely, they do not have full knowledge of the inputs and outputs. In this paper, we propose different attack methods considering various adversarial background knowledge. They are characterised by three dimensions as below:

Nodes' Attributes X of the Target Training Graph. This characterises how much the attacker knows about the attributes X of the nodes V in the graph G used to train the target model. Generally, the attacker can have full access to the attack nodes they obtain, and can directly collect the node attributes for applications that store them in each node, e.g., users' profiles in a social network service are accessible by end users. On the contrary, node attributes may not be obtained in some classification tasks. Note that, the attributes of other nodes which are not compromised should be invisible to the attacker consistently [46, 47]. Therefore, we consider that the attacker cannot obtain the attribute knowledge except the attack nodes.

Graph Structure A of the Target Training Graph. This characterises how much the attacker knows the graph structure of the target graph G. Unlike the attributes which contain only the information from one node, the graph structure presents the relationship among multiple nodes. An attacker knowing the edges of the attack nodes can construct a sub-graph consisted of both the attack nodes and their neighbours. Besides, while the node attributes are considered as private data, the connections such as friendships and following relations can be public. The attacker can reconstruct the target graph by crawling such public information [3, 6]. He can also utilise graph structure reconstruction methods [11, 20] to obtain this knowledge.

Shadow Dataset $G' = (V', E', X')$. This represents a dataset in the same domain as the target dataset. An example could be a scenario when the target dataset and shadow dataset are from the same large network but different sub-graphs or communities [8, 14]. In practice, the model owner may only have the privilege or the capability to train their model based on the sub-graph in an extensive network. We assume the attacker may also have this privilege for another sub-graph as prior attack settings [16, 20, 39].

4.3 Attack Taxonomy

Combining the above three dimensions, the knowledge of the attackers can be denoted as (X, A, G'). Based on whether the attacker has or has no knowledge of each item, we categories seven attacks by considering among the total eight cases. Note that we do not consider the case where the attacker has knowledge of neither three dimensions. The settings and scenarios for each of them are summarised as Table 2.

Attack-0$(X, A, *)$**:** We present our first attack when the attacker has access to the attributes, connections, but no a shadow dataset of the target GNNs. Here, we assume a more practical setting, where the attacker has only partial knowledge of the node attributes and their connections. Specifically, only the attributes and neighbour sub-graph structure of the attack nodes are known by the attacker. In practice, it considers the scenarios when the attacker can obtain some attack nodes among the target graph. For example, the attacker can create several accounts in a financial credit network so they can know their own profiles (attributes) and transactions to others (connections to neighbour). As for the setting where the attacker has full knowledge of all the node attributes or complete knowledge of the graph structure, is a stronger adversarial assumption than Attack-0.

Attack-1$(X, *, *)$**:** We now consider a more strict case when the attacker has access to the attributes, but neither access to the connections nor a shadow dataset of the target GNNs. In practice, this may represent the scenarios when the attacker can not fully control the attack nodes. For example, he can not create his own malicious accounts but obtain limited information (i.e. only the profile rather than the transactions) by exploiting the vulnerability of the financial credit systems. Namely, only the attributes of the attack nodes (partial knowledge of the node attributes) are obtained by the attacker.

Attack-2$(*, A, *)$**:** Similar to Attack-1, we consider another case when the attacker has access to the connections but no the attributes or the shadow graph. Such type of attack can be used in the GNN systems where the connections are public but the node attributes are private. For example, in a social network, the profiles for each users (node attributes) are private data which are protected, while the relationship among the users (connections) are often public which are easier to obtain. The attacker can manage to crawl the information from the public data and reconstruct the entire graph connections.

Attack-3$(*, *, G')$**:** We then consider a scenarios when the attacker only obtain a shadow graph data. As we introduced in Section 4.2, the shadow graph can be a different sub-graph or community from the same entire graph as the target graph. In practice, this represent when the attacker have overall knowledge about the target training graph or only have limited privilege or capability on an extensive network the same as the target GNN developer.

Attack-4(X, A, G')**:** This type of attack consider the scenario which is the combination in Attack-3 and Attack-0. The attacker can have access to both the attributes and neighbour connections to the attack nodes in the target graph, as well as the shadow dataset. In practice, this represents when the attacker successfully obtain the attack nodes while having knowledge on a shadow graph.

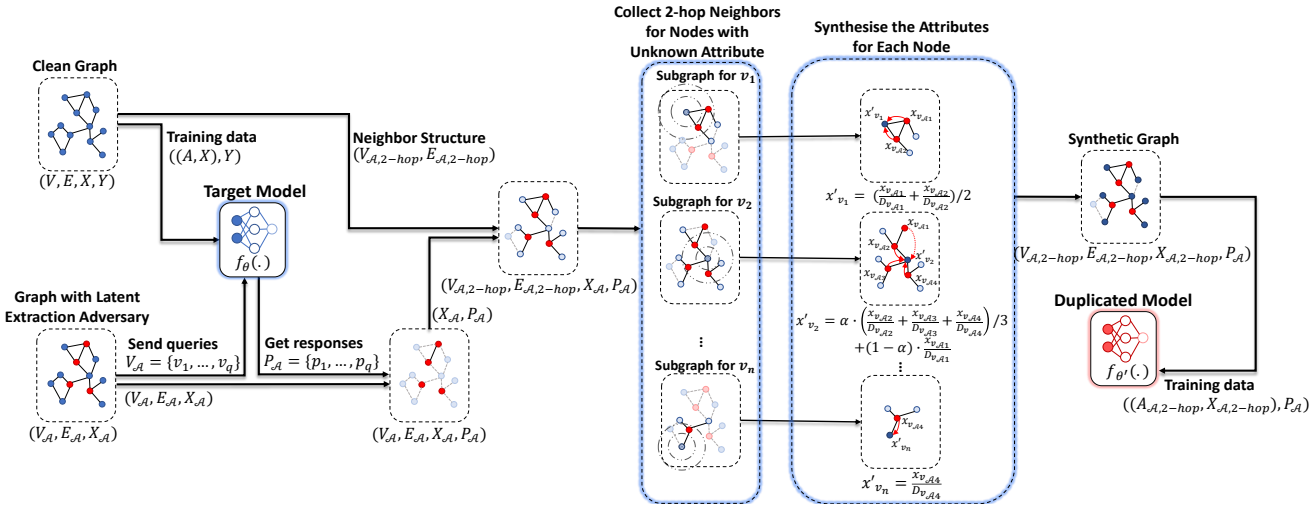


Figure 2: Illustration of Attack-0. After obtaining the query responses $P_{\mathcal{A}}$ from the target model f_θ, and the neighbour connections $(V_{\mathcal{A},2-hop}, E_{\mathcal{A},2-hop})$ from the target graph, the attacker synthesises the attributes $X'_{\mathcal{A},2-hop}$ for the neighbour nodes of the attack nodes. The combination of the attack nodes with known attributes and labels, and the synthetic nodes with generated attributes $(V_{\mathcal{A},2-hop}, E_{\mathcal{A},2-hop}, X'_{\mathcal{A},2-hop}, P_{\mathcal{A}})$ are used to train the duplicated GNN model $f_{\theta'}$ via semi-supervised learning.

Attack-5$(X, *, G')$**:** We then consider the scenario which is the combination in Attack-3 and Attack-1. The attacker can have access to only the attributes and the shadow dataset, but not the connections. Again we assume that the attacker can only obtain the attributes of the attack nodes.

Attack-6$(*, A, G')$**:** We finally consider the case which is the combination in Attack-3 and Attack-2. The attacker can have access to only the connections and the shadow dataset, but not the attributes.

Remark: Note that there is a cause where the attacker has no information about the attributes, connections, or task domain of the target dataset (i.e., the knowledge is represented as $(*, *, *)$). In this case, neither the input graph nor the model parameters are known to the attacker, while only the final prediction labels are exposed. As a result, the attacker is not able to gain any information regarding the inputs of the target GNNs, thus making it difficult to discover a link between inputs and outputs. In this paper, we do not focus on extraction attacks based on this assumption, and leave it as an open problem for future work.

5 ATTACK REALISATION

5.1 Attack-0

We first consider a scenario where the attacker obtains a set of attack nodes $V_{\mathcal{A}}$ and has both access to their attributes $X_{\mathcal{A}}$ and neighbour sub-graph structure $A_{\mathcal{A},k-hop}$. These attack nodes are randomly chosen among the total node set V to imitate the real-world scenarios where every node in the victim graph can be a potential attack node.

To extract the target models, the attacker intends to generate a graph for the duplicated model training. We call it *an attack graph* in the rest of our paper. The attack graph consists of the node attributes, graph structure, and node labels. The attacker attempts to obtain or generate the above items based on their adversarial knowledge. Specifically, the attack graph for the extracted model training can be built by three steps gathering each of the above items. Figure 2 shows a procedure for getting this attack graph.

Issuing queries and obtaining labels. In our assumptions, the attackers can obtain the attribute and query results of the attack nodes. The results of the response queries from the attack nodes can be considered as their node labels. Hence, they are utilised as the labelled nodes with known attributes to train a duplicated model for the node classification task.

Gathering neighbour connections. Knowing input attributes, output predictions, and connections among the attack nodes, the attacker can naturally employ supervised learning to train the duplicated model. However, the predictions of our attack nodes are also affected by their neighbours. Training the model by the attack nodes isolated among the graph will desert the impacts from the neighbours of the attack nodes and reduce the attacker performance. Therefore, our design should rationally consider the attributes of the neighbours around these nodes. Specifically, the attacker will gather the connections among the attack nodes and their neighbours, which are considered as the graph structure of the attack graph.

Synthesising attributes for in-accessible nodes. In our assumptions, the attacker only knows the attributes and query results of the attack nodes. Thus, the attacker needs to synthesise the attributes for the neighbour nodes with unknown attributes. In practice, most nodes have similar attributes as their neighbours [7, 16, 31]. Based on this observation, the synthetic attributes can be the combination of their neighbour nodes' attributes. Formally, to synthesise the attributes of a target node x'_{v_i}, the attacker first gathers all its known-feature neighbours, including n 1-hop nodes $\{v_{1,1-hop}, \ldots, v_{n,1-hop}\}$ and m 2-hop nodes $\{v_{1,2-hop}, \ldots, v_{m,2-hop}\} \subset V_{\mathcal{A}}$. For each of them,

Algorithm 1 Algorithm for Attack-0

Input:
q attack nodes' attributes $X_{\mathcal{A}} = \{x_{v_1}, x_{v_2}, ..., x_{v_q}\}$, q attack nodes' query results $P_{\mathcal{A}} = \{p_{v_1}, p_{v_2}, ...p_{v_q}\}$, graph structure of the 2-hop neighbour nodes set of the attack nodes $(V_{\mathcal{A},2-hop}, E_{\mathcal{A},2-hop})$, adjustment factor α.
Output:
Extracted Model $f_{\theta'}(.)$.

Generate adjacency matrix $A_{\mathcal{A},2-hop}$ for $(V_{\mathcal{A},2-hop}, E_{\mathcal{A},2-hop})$
Initialise a empty attribute set $X'_{\mathcal{A},2-hop}$
for $v_i \in V_{\mathcal{A},2-hop}$ **do**
  if $v_i \in V_{\mathcal{A}}$ **then**
    \\ Keep the attribute of attack nodes
    Collect x_{v_i} from $X_{\mathcal{A}}$
    Add x_{v_i} to $X'_{\mathcal{A},2-hop}$
    Label v_i as y'_{v_i} according to p_{v_i} in $P_{\mathcal{A}}$.
  else
    \\ Gather knowledge from the 1-hop neighbours
    Initialise a empty attribute set $X'_{v_i,1-hop}$
    for $v_j \in V_{v_i,1-hop}$ **do**
      if v_j is in $V_{\mathcal{A}}$ **then**
        $x'_{v_j} = x_{v_j}/D_{v_j}$
        Add x'_{v_j}/D_{v_j} to $X'_{v_i,1-hop}$
    \\ Gather knowledge from the 2-hop neighbours
    Initialise a empty attribute set $X'_{v_i,2-hop}$
    for $v_j \in V_{v_i,2-hop}$ and $\notin V_{v_i,1-hop}$ **do**
      if v_j is in $V_{\mathcal{A}}$ **then**
        $x'_{v_j} = x_{v_j}/D_{v_j}$
        Add x'_{v_j}/D_{v_j} to $X'_{v_i,2-hop}$
    \\ Synthesise attribute based on the 2-hop neighbours
    $x'_{v_i} = \alpha \cdot \text{mean}(X'_{v_i,1-hop}) + (1-\alpha) \cdot \text{mean}(X'_{v_i,2-hop})$
    Add x'_{v_i} to $X'_{\mathcal{A},2-hop}$
Train 2-layer GCN $f_{\theta'}(.)$ based on $(X'_{\mathcal{A},2-hop}, A_{\mathcal{A},2-hop}, Y_{\mathcal{A}})$

the impact to the targets can be represented as $v_{j,k-hop}/D_j$, where D_j represents the degree of this neighbour node v_j. Considering an adjustment factor α to balance the effects from one or two hops nodes, the attacker synthesises the feature of the target node as:

$$x'_{v_i} = \alpha \sum_{j=1}^{n} \frac{x_{v_j,1-hop}}{nD_j} + (1-\alpha) \sum_{j=1}^{m} \frac{x_{v_j,2-hop}}{mD_j} \quad (2)$$

Learning the extracted model. After generating the attributes for these nodes, the attacker can obtain a graph that includes all attack nodes and their neighbours with the known or synthetic attributes, and then train a node classification GNN model as the extracted model. Note that the attacker does not label the synthetic nodes. Unlike the labels of the attack nodes that come from the query responses, the synthetic nodes are inaccessible to the attacker. He can neither modify their attributes nor send queries to the target models. As a result, only the attack nodes can be labelled and the extracted models are trained via semi-supervised learning. The overall process of the Attack-0 is shown as Algorithm 1.

5.2 Attack-1

We then intensify the restriction to the attacker and consider the case when the attacker has only knowledge about the attributes of the attack nodes $X_{\mathcal{A}}$. For this type of attack, the attacker also needs to first generate an attack graph for the extracted model training. Compared with Attack-0, since the graph structure is unknown, the attacker needs to generate the connections between the nodes. A procedure of the attack is shown in Figure 3 and the attack graph is generated as follows.

Issuing queries and obtaining labels. Similar to Attack-0, the attributes and the query responses of the attack nodes can be used as the labelled nodes in the attack graph.

Synthesising connections among attack nodes. Different from Attack-0, the graph structure is unknown to the attacker. If all the attack nodes are deemed to be isolated, the impacts from the neighbours of the attack nodes cannot be taken into considerations. To solve this problem, the attacker needs to construct a substitute graph based on the known attributes.

Generally, the attributes of nodes in a graph and the connections among them are tightly correlated [1, 45]. Thus, it is possible to infer or reconstruct the graph structure based on the node attributes. Based on this intuition, several prior studies about graph synthesis and generation have been developed to generate graphs [12, 30, 49]. Among others, we use a graph generation method called Learning Discrete Structures (LDS) [12]. It can generate the graphs by considering their performance on classification problems, which meets the tasks of our target models. Therefore, given the attributes of the attack nodes, the attacker can synthesise the connections among them and use the synthetic structure as the attack graph.

Learning the extracted model. After the above steps, the attacker can obtain a substitute graph with attack node attributes, corresponding prediction labels, and a generated graph structure. Then he can use supervised learning to train the duplicated models. Note that, due to the approximation of the edges distributions, the density of the generated graph can be set close to the target. Hence, most of the nodes generated via this method are not isolated and the attacker does not need to synthesise their neighbours as Attack-0.

5.3 Attack-2

We consider another scenario when the attacker obtains the entire graph structure knowledge A while having no access to any nodes in V even for the attack nodes $V_{\mathcal{A}}$. Namely, he cannot obtain the node attributes X. As a result, he needs to build the attack graph by synthesising the attributes, as depicted in Figure 4. The detailed steps are:

Issuing queries and labelling the attack nodes. Even though the attacker has no access to the node attributes, he can still obtain the responses and use them to label the attack nodes. After generating attributes, these labelled nodes can be used during the extracted model training.

Gathering the target graph as attack graph. To reconstruct the target model, the attacker naturally utilises the entire known graph structure to build the attack graph.

Assigning one-hot vectors as node attributes. Without any knowledge about the attributes, the attacker proposes to first synthesise them and build a surrogate training graph. As discussed in

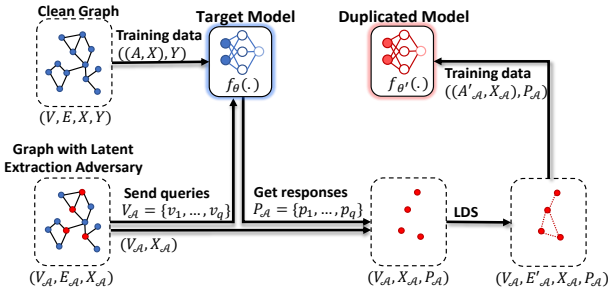


Figure 3: Illustration of Attack-1. After obtaining the query responses $P_{\mathcal{A}}$ from the target model f_θ, the attacker can only obtain discrete nodes with their attributes $X_{\mathcal{A}}$. A synthetic graph can be generated based on these attributes via graph generation method LDS [12]. Then the attack nodes with attributes and labels and the synthetic graph structure $(V_{\mathcal{A}}, E'_{\mathcal{A}}, X_{\mathcal{A}}, P_{\mathcal{A}})$ are used to train the duplicated GNN model $f_{\theta'}$ via supervised learning.

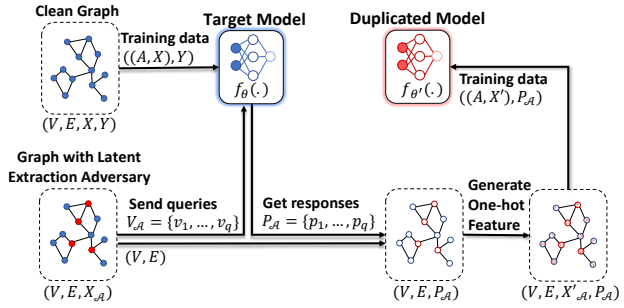


Figure 4: Illustration of Attack-2. After obtaining the query responses $P_{\mathcal{A}}$ from the target model f_θ, the attacker can only obtain the graph $(V, E, P_{\mathcal{A}})$ without node attributes. The attacker assigns one-hot vectors to every node as their synthetic attributes X' based on their graph index. Then the attack nodes with attributes and labels and the synthetic graph structure $(V, E, X', P_{\mathcal{A}})$ are used to train the duplicated GNN model $f_{\theta'}$ via semi-supervised learning.

Section 4, the attribute of a node is also related to the entire graph structure and its position in it. To synthesise attributes associated with the structure knowledge, the attacker uses the index of the nodes to generate *one-hot vectors* as their attributes. For example, the attribute of v_1 will be $[1, 0, 0, ..., 0]$ while for v_2 is $[0, 1, 0, ..., 0]$. These attributes represent the identity of the nodes and contain information about the graph structure.

Note that, generating arbitrary features does not satisfy the model training with graph structure since they might bear no resemblance to their actual attributes. Meanwhile, it is also hard to reconstruct the original attributes of the target graph. As mentioned, the node attributes and the graph structure are tightly related. The objective of our attacks is to extract the mapping from node attributes to the node labels based on the graph structure. Thus, inferring the original attributes of the graph based on the graph structure and the node label or even only the node position can be considered as the reversed function of our target models. It is difficult to first learn this reversed mapping and then extract the target models.

Learning the extracted model. After synthesising the attributes, the attacker can construct a surrogate model by these attributes, prediction labels of the attack nodes, and the entire graph structure via semi-supervised learning. Different from previous attacks, the inputs of the surrogate models are the *one-hot vectors*. As a result, the nodes will be inferred via their indexes in the graph. Since the entire target graph structure is utilised, the extracted models can be used to classify all the nodes in the target graph as the target model.

5.4 Attack-3

We now consider the case when the attacker does not know the node attributes and their connections. But we assume the attacker has access to a shadow graph $G' = (V', E', X')$ defined in Section 4. Under this adversarial assumption, the attacker has no knowledge about the target graph. Therefore, the extraction can only refer to the shadow graph. As introduced in Section 4, the shadow graph has the same domain as the target. Therefore, it is possible to utilise the knowledge from a shadow graph, i.e., using it as the attack graph.

Specifically, the attacker first gathers both the node attributes X' and the graph structure A' of a shadow graph. He can also obtain the corresponding labels Y' for some nodes in the shadow graph. Then, this shadow dataset $D' = (X', A', Y')$ can be used to train a surrogate model via semi-supervised learning. Since the dimensions of the node attributes from the graph with the same domain are also the same, the parameters of the surrogate model (the weights W) have the same size. Therefore, these weights can be used as the extracted model which achieves similar functionality as the target model if the target and shadow graphs are in the same domain.

5.5 Attack-4

In this attack, the attacker is assumed to have access to the attack nodes $V_{\mathcal{A}}$ as Attack-0. Besides, he can collect a shadow graph G' as Attack-3. With both the background knowledge as Attack-0 and Attack-3, the attacker proposes to combine them together. In particular, an associated attack graph is built by combining the attack graphs for these two attacks.

The attacker first generates an attack graph consisting of the attacker and synthetics nodes with the same strategy as Attack-0. Then, the shadow graph is set to be the second attack graph as Attack-3. Since the attack nodes in the first attack graph are often not connected to the second attack graph, the attacker will not synthesise the connections between them. It can avoid the negative impacts among the attack nodes and the shadow graph. Hence, the associated attack graph consists of these two isolated graph components. After that, the attacker can train the extracted model on this associated attack graph based on all the known node attributes, their graph structure, and the labelled nodes from both the shadow graph and the attack nodes in the target graph.

Algorithm 2 Algorithm for Attack-6

Input:
Shadow graph $G' = (V', E', X')$, graph structure (V, E),
Output:
Extracted Model $f_{\theta'}(.)$.

\\ Extract GCN as Attack-2 on target graph
Generate one-hot vector attributes $X_{one-hot}$ from (V, E).
Train GCN $f_{attack_2}(.)$ based on $(V, E, X_{one-hot}, Y)$.
\\ Extract GCN as Attack-2 on shadow graph
Generate one-hot vector attributes $X'_{one-hot}$ from (V', E').
Train GCN $f'_{attack_2}(.)$ based on $(V', E', X'_{one-hot}, Y')$.
\\ Extract GCN as Attack-3 on shadow graph
Train GCN $f'_{attack_3}(.)$ based on (V', E', X', Y').
\\ Posterior feature generation
for $v'_i \in V'$ **do**
    $x'_{attack_2,v'_i} = f'_{attack_2}(v'_i)$
    $x'_{attack_3,v'_i} =, f'_{attack_3}(v'_i)$.
Stack X'_{attack_2} and X'_{attack_3} as X'_{attack_6}.
\\ Build ensemble models
Train DNN $f_{attack_6}(.)$ based on $(V', E', X'_{attack_6}, Y')$.
$f_{\theta'}(.) = f_{attack_6}(f_{attack_2}(.), f'_{attack_3}(.))$

5.6 Attack-5

This attack considers the case where the attacker has access to a shadow graph G' and also the attributes of the attack nodes $X_{\mathcal{A}}$ in the target graph. The adversarial knowledge is from both Attack-1 and Attack-3. Similar as Attack-4, the attacker can combine them to utilise the background knowledge of $X_{\mathcal{A}}$ and G' in this attack.

To implement the attack, a graph generation method is used to construct a structural graph for the attack nodes based on their attributes as Attack-1. The graph structure and the corresponding nodes with attributes and query responses consist of the first attack graph. The attacker again uses the shadow graph as the second attack graph. Due to the same reason as Attack-3, the attacker will not synthesise connections between two attack graphs. Then, two attack graphs are associated to build the combined attack graph for the training of the extracted model.

5.7 Attack-6

This attack considers the assumption that the attacker has no access to the node attributes but the entire graph structure knowledge A and a shadow graph G'. Compared with Attack-2, the attacker can gather extra knowledge from the shadow graph. However, it is hard to directly utilise this node attribute knowledge and implement a similar design as Attack-2. In Attack-2, the attributes of the nodes are one-hot vectors corresponding to the node indexes. Thus, the dimension of the synthetic attributes will be the same as the number of nodes which does not match the original attributes.

To combine Attack-2 and 3, the attacker can build the ensemble models. Two models utilising different background knowledge via the methods as Attack-2 or Attack-3 are trained separately. Even though their inputs are different, both their outputs are the posteriors of the label of nodes. An attack model is trained to predict the final labels based on two posteriors. Specifically, the inputs of the attack model are the stack of the outputs from the two extracted models while the outputs are the final prediction labels. Since the attacker cannot obtain the posteriors from the target models of our attacks, the attack models are first generated in the shadow graph. The detailed processes are:

Extracting models on the shadow graph. We extract a model on the shadow graph via Attack-2 by only using its graph structure knowledge. The inputs of this model are one-hot vector attributes. Then, we extract another model on the shadow graph via Attack-3 by using its entire graph data.

Training an attack model. Since the attacker can obtain all knowledge about the node attributes, graph structure, he can feed them into the two models generated above and gather their output posteriors. Then the posteriors and their corresponding labels are used to train a simple MLP model to predict the final labels.

Extracting the model on the target graph. After building the attack model on the two emulated models in shadow graph, the attacker can generate the real extracted models. To utilise the structure knowledge of the target graph, the model is extracted via Attack-2.

Building the ensemble models. After training the two extracted models for Attack-2, Attack-3 and the attack model, the attacker can set up the ensemble model. The output posteriors of the two extracted models are fed into the attack model to generate the final predictions.

Datasets	Node Number	Edge Number	Class Number
Cora	2708	5429	7
Citeseer	3327	4732	6
Pubmed	19717	44338	3

Table 3: Dataset Statistics

6 EXPERIMENTS

In this section, we present a comprehensive set of experiments to evaluate our attacks. We first introduce the experiment setting and then present the detailed results for each attack.

6.1 Experimental Setup

Datasets. Three public datasets are used to evaluate our proposed attacks, including Cora, Citeseer, and Pubmed [25]. All of them are benchmark datasets that are widely used for the evaluation of node classification models. These three datasets are citation networks whose nodes represent the publications and edges are their citations. The detailed statistics of the datasets are shown in Table 3.

Datasets Configuration. We configure the datasets for our different attacks. For Attack-0, Attack-1, and Attack-2 which do not contain the knowledge about the shadow dataset, we use the entire graph data to train the target models. For the Cora dataset, we split the network into 140 (about 5% of the total nodes) labelled nodes as training part, 300 (11%) labelled nodes as validation, and the rest of them unlabelled. Among the unlabelled nodes, we choose 1000 (37%) of them as the testing sets. For the Citeseer, we set 120 (4%)

Metrics	Accuracy			Fidelity		
Dataset	Cora	Citeseer	Pubmed	Cora	Citeseer	Pubmed
Target Model	0.816	0.713	0.800	–	–	–
Simple DNN (baseline)	0.577 ± 0.004	0.596 ± 0.004	0.727 ± 0.006	0.590 ± 0.010	0.632 ± 0.005	0.761 ± 0.005
Attack-0	**0.799 ± 0.009**	0.684 ± 0.016	0.736 ± 0.004	**0.896 ± 0.008**	**0.848 ± 0.019**	**0.890 ± 0.007**
Attack-1	0.798 ± 0.006	**0.708 ± 0.007**	**0.751 ± 0.003**	0.825 ± 0.007	0.754 ± 0.005	0.857 ± 0.003
Attack-2	0.762 ± 0.012	0.548 ± 0.004	0.652 ± 0.036	0.809 ± 0.006	0.602 ± 0.003	0.728 ± 0.035
Target Model	0.816	0.697	0.806	–	–	–
Attack-3	0.809 ± 0.007	0.692 ± 0.004	0.799 ± 0.001	0.790 ± 0.005	0.714 ± 0.002	0.818 ± 0.009
Attack-4	0.801 ± 0.009	**0.708 ± 0.002**	**0.800 ± 0.008**	0.790 ± 0.011	**0.736 ± 0.008**	**0.837 ± 0.004**
Attack-5	**0.832 ± 0.004**	0.699 ± 0.001	0.799 ± 0.002	**0.807 ± 0.002**	0.727 ± 0.002	0.818 ± 0.003
Attack-6	0.800 ± 0.017	0.649 ± 0.017	0.737 ± 0.092	0.791 ± 0.019	0.731 ± 0.019	0.813 ± 0.086

Table 4: Model accuracy/fidelity for all attacks on three different datasets. Attack-0, Attack-1, and Attack-2 target at the model trained in the entire dataset. Attack-3, Attack-4, Attack-5, and Attack-6 target the model trained in a sub-graph split from the entire graph. Best results are highlighted in bold.

labelled nodes as training part, 500 (15%) labelled nodes as validation, and 1000 (30%) unlabelled nodes as the testing set. And for the Pubmed dataset, 60 (0.3%) labelled nodes are used as the training part, while 500 (2.5%) labelled nodes as validation. We again choose 1000 (5.1%) unlabelled nodes as the testing set.

For Attack-3, Attack-4, Attack-5, and Attack-6 using shadow dataset, we split the network into two parts: the graph for target model training and the graph assumed to be known by the attacker. To generate the shadow dataset, we first split the entire network into several communities by Clauset-Newman-Moore greedy modularity maximisation [9], and then divide them into two datasets. For the Cora network, we generate the training graph for the target models which consists of 1408 (about 50%) nodes. For the Citeseer, the training graph for the target models has 1320 (about 40%) nodes. And for the Pubmed dataset, we set the training dataset for the target models to be the graph with 1408 (about 50%) nodes. The rest of them which consist of 1300 nodes are used as the shadow dataset. We split both the target and the shadow networks into training and testing parts. Other configurations for the datasets share the same settings as the datasets for Attack-0, Attack-1, and Attack-2.

Evaluation Metric. We evaluate our attacks from two aspects based on the two different definitions about the similar performance of extracting the models following the evaluation methods of prior extraction attacks in DNNs [13, 36]. The first one is *fidelity* which evaluates how similar the surrogate models and the target models are. Specifically, it is defined as the percentage of the v_i in V where $f_{\theta'}(v_i) = f_{\theta}(v_i)$. It is calculated by dividing the number of common predictions between two models by the number of the total testing inputs. For higher fidelity, the extracted models are expected to have more similar performance as the target models. Extracted models with high fidelity can be used when the attacker requires further analysis about the target models, e.g., being used in adversarial attacks as a target with infinite queries. Another metric is the *accuracy* that represents how accurate the surrogate models are in testing data. Specifically, it is the percentage of the v_i in V where $f_{\theta'}(v_i) = y_{v_i}$. It is calculated by dividing the number of the correct classified nodes by the number of the total testing nodes. Extracting models with higher accuracy allows the attacker to directly use them for the inference in the target application tasks rather than querying the target models, injuring the interests of model owners.

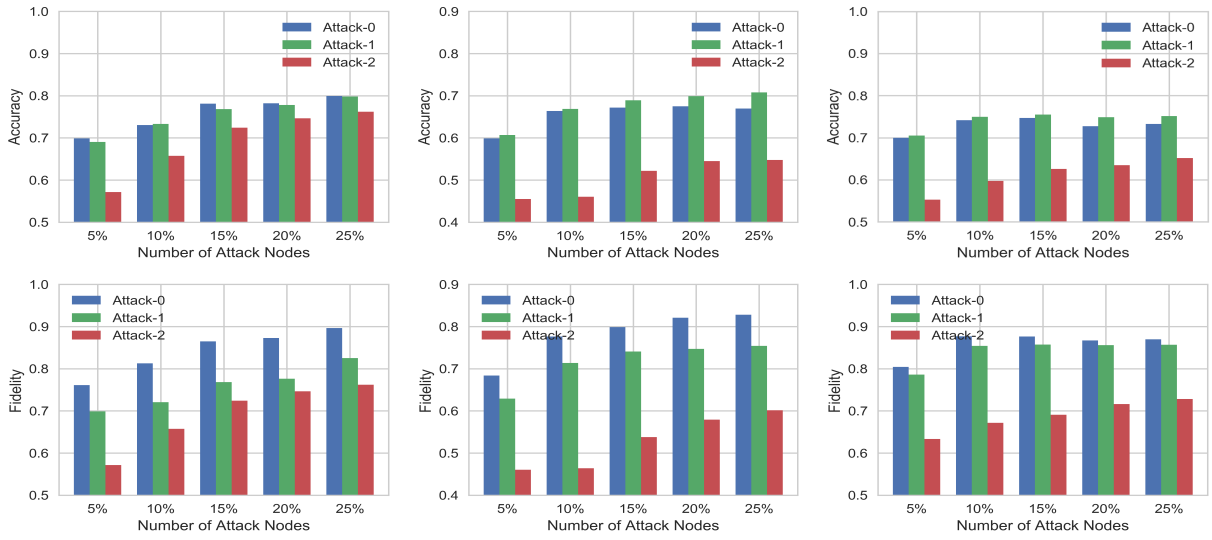
Models. Our experiments consider the case where the target model is a 2-layer graph convolution network, introduced in Eq. 1. The number of features in the hidden layer is 16. The activation function for the hidden layer is ReLU and for the output layer is softmax. We also apply a dropout layer with a 0.5 dropout rate after the hidden layer. We use the Adam optimiser with a learning rate of 0.02 and training epochs of 200. The loss function of our model is negative log-likelihood loss.

6.2 Attack Performance

Overview. Table 4 shows an overview of the performance of our seven attacks. For Attack-0, Attack-1 and Attack-2, the numbers of the attack nodes obtained by the attacker are chosen to be about 25% of the total nodes in the target networks. For Attack-3, Attack-4, Attack-5, and Attack-6, the size of the shadow graph is set to be almost the same as the target, and the attacker is assumed to obtain fewer nodes which are about 10%. It can be found that our attacks achieve nearly equal accuracy as the target model as the baseline accuracy. Meanwhile, most of our attacks gain about 80% fidelity, which means that our extracted models mostly predict the inputs as the targets. We highlight the attacks with the best performance among others. Detailed discussions for each attack are presented as follow.

Attack-0. Attack-0 is shown to achieve the highest fidelity since their training data is the most similar to the target model. We analyse their performance by adjusting several factors in the design.

Figure 5 shows the relationship among the number of the attack nodes and the fidelity/accuracy of the surrogate models from 5% of the total nodes to 25%. For larger numbers of attack nodes, both accuracy and fidelity increase. The accuracy of the extracted model achieves about 79.9% in the Cora dataset which is very close to the target model 81.5%. And the fidelity of the duplicated model is about 90%. For the Citeseer, the accuracy increases from 59.9% to 67.0% when obtaining the attack nodes from 5% to 25%. The



(a) Cora (b) Citeseer (c) Pubmed

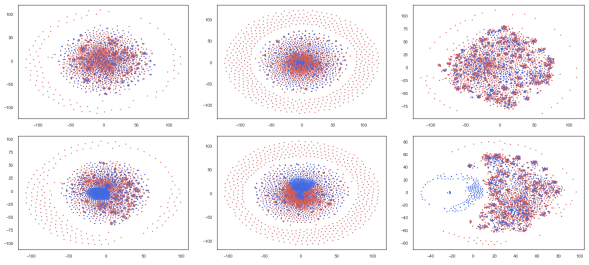
Figure 5: Impact of the number of the attack nodes in Attack-0, Attack-1, and Attack-2

accuracy with about 25% nodes is close to the baseline accuracy 70.0%. The fidelity reaches 82.8% when the number of the attack node is about 25% of the nodes in the total graph. When attacking the model trained in Pubmed, the accuracy of the duplicated model increases from 70% to 73%.

We also evaluate how synthesising the neighbours affects our attack performance. Table 5 shows the accuracy and fidelity of the attack with and without the synthetic nodes. It can be found that, synthesising the attributes for the neighbours of the attack nodes can improve the fidelity of our attacks. We also evaluate the attack performance when synthesising more neighbour nodes. It is shown that too many synthetic nodes will hurt our attacks. We compare the feature distribution of the graph generating by different strategies in Figure 6. It is shown that the graph generated by synthesising only the first order achieves the most similar distribution as the target graph that matches our attack results. We also compare the degree distribution in Appendix A.1.

We now discuss the impact of the adjustment factor α. Figure 7 shows both the accuracy and fidelity of the attack with variant α. The experiments show that this factor does affect the attack performance but mostly inside ±5%. We can also find that the attack performance raises when α increases for both Cora and Citeseer. Larger α means the synthetic attributes of the nodes are more based on their 1-hop neighbours. This is reasonable since the relationship between the synthetic nodes to their 1-hop neighbours is stronger than the 2-hop neighbours. Meanwhile, the performance from Pubmed is undulate. To achieve the best attack performance, the attacker can carefully choose the adjustment factor by considering the characteristic of the graph.

Attack-1. In Attack-1, only the attributes of the attack nodes are known to the attackers while their connections are unknown. Therefore, we generate the graph structure based on these node features. Figure 5 shows the relationship between the number of attack nodes and the attack performance. Similar to Attack-0, more attack nodes can significantly increase the accuracy and fidelity of the extracted models.



(a) Cora (b) Citeseer (c) Pubmed

Figure 6: Feature distribution of the nodes including only first-order synthetic neighbour nodes (Upper) and both first and second-order (Lower) for Attack-0 projected into a 2-dimension space using t-SNE.

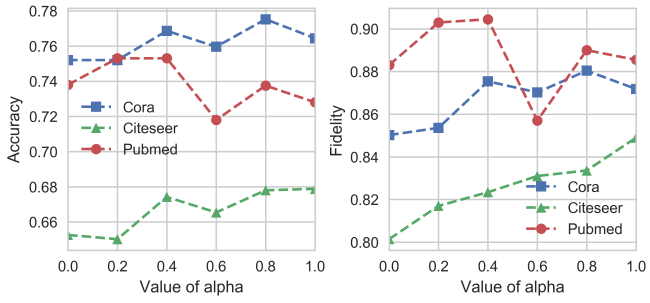


Figure 7: Impact of the adjustment factor α for Attack-0

Metric	Dataset	Without Synthetic	First Order Neighbour Synthetic	Second Order Neighbour Synthetic
Accuracy	Cora	0.797 ± 0.012	**0.799 ± 0.009**	0.793 ± 0.008
	Citeseer	**0.688 ± 0.013**	0.684 ± 0.016	0.681 ± 0.017
	Pubmed	0.735 ± 0.027	**0.736 ± 0.004**	0.731 ± 0.013
Fidelity	Cora	0.869 ± 0.012	**0.896 ± 0.008**	0.889 ± 0.009
	Citeseer	0.816 ± 0.030	**0.848 ± 0.019**	0.834 ± 0.015
	Pubmed	0.879 ± 0.030	**0.890 ± 0.007**	0.886 ± 0.015

Table 5: Impact of the synthetic nodes for Attack-0. Best results are highlighted in bold.

To clearly show how the graph structure generation contributes to our design, we set a baseline, which uses deep neural networks to infer the output labels based only on the input features. The results for both accuracy and fidelity of the surrogate models in three different datasets are shown in Table 6. In the Cora dataset, our design achieves about 79.8 for the accuracy of the extracted model while DNNs have only 57.7%. And our attack improves the fidelity from 59.0% to 82.5%.

To show how the generated graph structure matches the original graph, we also evaluate the degree distribution of the graph generated by our attack methods. Notice that the attribute distribution should be very similar to the target graph since the attributes in Attack-1 are all from the attack nodes which is considered as a sample of the original. The comparisons between the degree distribution of the generated graph and the target graph for three datasets are shown in the Appendix A.2. We show that they are more similar comparing with the distribution without graph generation method. This demonstrates that using graph structure generation is a good approach to help reconstruct the graph structure and further improve our attack.

Attack-2. This attack considers the scenario when the attacker has only knowledge about the graph structure. The results for both accuracy and fidelity are shown in Table 4. And Figure 5 shows how the number of the attack nodes affects our attack. Similarly, both accuracy and fidelity increase when obtaining more attack nodes.

Notice that, Attack-2 has the worst performance comparing with Attack-0 and Attack-1. This might cause by the less similarity of the synthetic items for the attack graph we generated. For other attacks, our design can generate node attributes or connections similar to the target graph. However, due to the lack of knowledge about the attributes, the synthetic attributes of the attack graph for this type of attack are one-hot vectors that are far from the target. If the attacker can obtain some knowledge about the node attributes which can be used to synthesise similar attributes, the performance can be improved.

Attack-3. For Attack-3, we consider the case where the attacker can only have a shadow graph defined in Section 4.2 without knowing both attributes and graph structures for the target models. The results for both accuracy and fidelity are shown in Table 4. Compared to previous attacks with some background knowledge of the target graph, the accuracy of this type of attack is similar or even better. Thus, obtaining the complete graph data can achieve high accuracy if the shadow graph has the same domain as the target. However, the fidelity of the attack is significantly smaller than previous attacks since the training graph data of the target

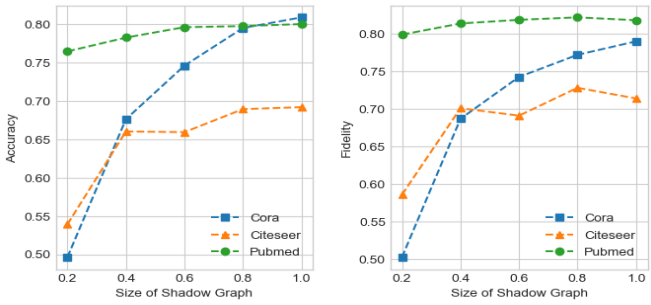


Figure 8: Impact of the shadow graph size for Attack-3

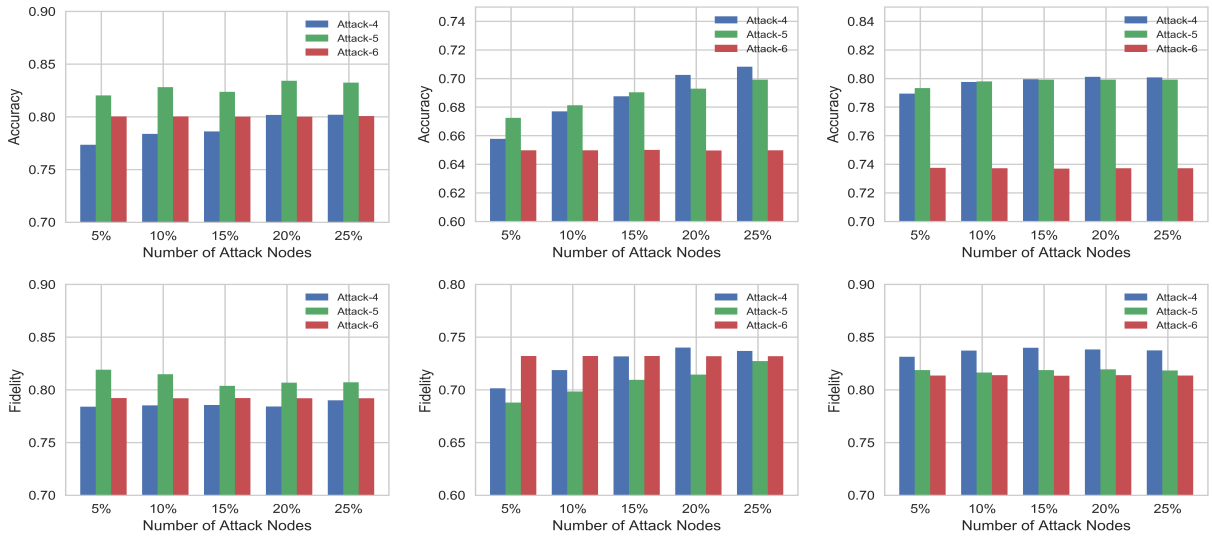
Metric	Dataset	Simple DNN	LDS-GNN [12]
Accuracy	Cora	0.577 ± 0.004	**0.798 ± 0.006**
	Citeseer	0.596 ± 0.004	**0.708 ± 0.007**
	Pubmed	0.727 ± 0.006	**0.751 ± 0.003**
Fidelity	Cora	0.590 ± 0.010	**0.825 ± 0.007**
	Citeseer	0.632 ± 0.005	**0.754 ± 0.005**
	Pubmed	0.761 ± 0.005	**0.857 ± 0.003**

Table 6: Fidelity/accuracy for Attack-1. Best results are highlighted in bold.

model is entirely different from our extracted graph. Our target model is built as the GCN model which is transductive, so it is hard to gain an extracted model with similar functionality.

We also analyse the effect for different knowledge of the shadow sub-graph. Figure 8 shows the relationship between the attack performance and the size of shadow graph. The x-axis represents the ratio of the size of the shadow graph to the target graph. It can be found that while knowing the larger size of the shadow graph, the accuracy of the surrogate models increases a lot. It is obvious since the attacker can extract more knowledge from a larger training graph. It can also be found that the fidelity of the surrogate models becomes saturated even the shadow graph size becomes larger. As discussed, the target GCN model is transductive, which makes the attacker difficult to obtain a highly equivalent model. Therefore, the fidelity of our attack will reach the ceiling when the size of the shadow graph keeps increasing.

Attack-4. Now we consider the case when the attacker has access to a shadow graph as well as some attack nodes in the target graph. Based on Table 4, it can be found that Attack-4 achieves higher accuracy and fidelity than Attack-3. It demonstrates that obtaining extra knowledge can lead to better attack performance. Note that,



(a) Cora (b) Citeseer (c) Pubmed

Figure 9: Impact of the number of the attack nodes in Attack-4, Attack-5, and Attack-6

the improvement of the fidelity is more significant than the accuracy especially for the Citeseer and Pubmed dataset. This shows that the knowledge from the target graph can be helpful to extract models with similar predictions as the target model.

We also evaluate how the number of the attack nodes affects the attack performance. Figure 9 shows the accuracy and fidelity for different numbers of attack nodes. Compared with Attack-0 which does not know the shadow graph, both accuracy and fidelity are only slightly affected by the numbers of the attack nodes. Note that, the number of the attack nodes and their 2-hop neighbours are commonly smaller than the number of nodes in shadow graph. Thus, learning from the combination of both of them, the performance of Attack-4 is hardly affected by the number of the attack nodes as the former attacks (Attack-0/1/2).

Attack-5. Now we consider the case when the attacker has access to a shadow graph and also some attack nodes in the target graph. Based on Table 4, it can be found that this attack achieves slightly higher accuracy and fidelity than the Attack-3 and nearly equal attack performance as the Attack-4. But since the knowledge of Attack-5 is less than Attack-4 (the neighbour graph structure is unknown), the fidelity is lower which is similar to the comparison between Attack-0 and 1. It demonstrates that obtaining more background knowledge can enhance our extraction attacks more. Similar to Attack-4, the number of the attack nodes can only slightly increases the accuracy and fidelity of the attacks, as shown in Figure 9.

Attack-6. Finally, we discuss the attack when the attacker has both knowledges about the graph structure and the shadow graph. With the help of shadow graph, the overall performance of Attack-6 is significantly higher than Attack-2. It demonstrates that introducing knowledge about the node attributes can improve the attack performance. It also achieves comparative performance as the Attack-4 and Attack-5.

When assessing the impact from the attack nodes, we find that increasing the number of the attack nodes can not affect the attack performance, as shown in Figure 9. This is caused by the different attack performance of the two component models in the ensemble models for Attack-6. Based on our design, the ensemble models infer the node label based on the posteriors of two extracted models. Since the component model built as Attack-3 achieves both higher accuracy and fidelity than the model for Attack-2, the ensemble models learn more from the model built as Attack-3. Using more attack nodes and increasing the accuracy of the model for Attack-2 can hardly impact the overall attack performance.

7 CONCLUSION AND FUTURE WORK

In this paper, we demonstrate a model extraction attack against GNNs. We first generate legitimate-looking queries as the normal nodes among the target graph, then utilise the query responses and accessible structure knowledge to reconstruct the model. We characterise the problem into seven threat models considering different knowledge of the attacker. Then we accordingly propose seven attacks based on the knowledge and the query responses. Our experimental results show that our attack obtains surrogate models with similar predictions as the targets.

An emerging research direction is a defence against the extraction attacks in GNNs. Existing defences against the model extraction attacks on DNN system propose to monitor and filter the input queries [23, 24], which can be extended and combined wit the structure analysis when implementing the defence for GNNs. We consider it as our future work.

ACKNOWLEDGMENT

This research was supported by the Australian Research Council (ARC) under a Future Fellowship No. FT210100097.

REFERENCES

[1] Deepak Bhaskar Acharya and Huaming Zhang. [n. d.]. Feature Selection and Extraction for Graph Neural Networks. In *Proc. ACM SE '2020.*

[2] Wu Bang, Yang Xiangwen, Pan Shirui, and Yuan Xingliang. 2021. Adapting Membership Inference Attacks to GNN for Graph Classification: Approaches and Implications. In *2021 IEEE International Conference on Data Mining (ICDM).* IEEE.

[3] Salvatore Catanese, Pasquale De Meo, Emilio Ferrara, Giacomo Fiumara, and Alessandro Provetti. [n. d.]. Crawling Facebook for social network analysis purposes. In *Proc. WIMS 2011.*

[4] Varun Chandrasekaran, Kamalika Chaudhuri, Irene Giacomelli, Somesh Jha, and Songbai Yan. 2018. Exploring connections between active learning and model extraction. *arXiv preprint arXiv:1811.02054* (2018).

[5] Heng Chang, Yu Rong, Tingyang Xu, Wenbing Huang, Honglei Zhang, Peng Cui, Wenwu Zhu, and Junzhou Huang. 2020. A Restricted Black-Box Adversarial Framework Towards Attacking Graph Embedding Models. In *Proc. AAAI.*

[6] Duen Horng Chau, Shashank Pandit, Samuel Wang, and Christos Faloutsos. [n. d.]. Parallel crawling for online social networks. In *Proc. WWW 2007.*

[7] Jinyin Chen, Xiang Lin, Ziqiang Shi, and Yi Liu. 2020. Link Prediction Adversarial Attack Via Iterative Gradient Attack. *IEEE Trans. Comput. Soc. Syst.* 7 (2020).

[8] Wei-Lin Chiang, Xuanqing Liu, Si Si, Yang Li, Samy Bengio, and Cho-Jui Hsieh. [n. d.]. Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks. In *Proc. KDD 2019.*

[9] Aaron Clauset, Mark EJ Newman, and Cristopher Moore. 2004. Finding community structure in very large networks. *Physical review E* (2004).

[10] Suguo Du, Xiaolong Li, Jinli Zhong, Lu Zhou, Minhui Xue, Haojin Zhu, and Limin Sun. 2018. Modeling Privacy Leakage Risks in Large-Scale Social Networks. *IEEE Access* 6 (2018), 17653–17665.

[11] Vasisht Duddu, Antoine Boutet, and Virat Shejwalkar. 2020. Quantifying Privacy Leakage in Graph Embedding. (2020). arXiv:2010.00906

[12] Luca Franceschi, Mathias Niepert, Massimiliano Pontil, and Xiao He. [n. d.]. Learning Discrete Structures for Graph Neural Networks. In *Proc. ICML 2019 (Proceedings of Machine Learning Research).*

[13] Chengsi Gao, Bing Li, Ying Wang, Weiwei Chen, and Lei Zhang. [n. d.]. Tenet: A Neural Network Model Extraction Attack in Multi-core Architecture. In *Proc. GLSVLSI '21: Great Lakes Symposium on VLSI 2021.* ACM, 21–26.

[14] Hongyang Gao, Zhengyang Wang, and Shuiwang Ji. [n. d.]. Large-Scale Learnable Graph Convolutional Networks. In *Proc. KDD 2018.*

[15] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. [n. d.]. Neural Message Passing for Quantum Chemistry. In *Proc. ICML 2017.*

[16] Neil Zhenqiang Gong and Bin Liu. [n. d.]. You Are Who You Know and How You Behave: Attribute Inference Attacks via Users' Social Friends and Behaviors. In *Proc. USENIX Security 16.*

[17] Neil Zhenqiang Gong and Bin Liu. 2018. Attribute Inference Attacks in Online Social Networks. *ACM Trans. Priv. Secur.* 21 (2018).

[18] Xueluan Gong, Qian Wang, Yanjiao Chen, Wang Yang, and Xinchang Jiang. 2020. Model Extraction Attacks and Defenses on Cloud-Based Machine Learning Models. *IEEE Commun. Mag.* 58, 12 (2020), 83–89.

[19] Payas Gupta, Swapna Gottipati, Jing Jiang, and Debin Gao. [n. d.]. Your love is public now: questioning the use of personal information in authentication. In *Proc. ASIA CCS '13.*

[20] Xinlei He, Jinyuan Jia, Michael Backes, Neil Zhenqiang Gong, and Yang Zhang. 2020. Stealing Links from Graph Neural Networks. (2020). arXiv:2005.02131

[21] Matthew Jagielski, Nicholas Carlini, David Berthelot, Alex Kurakin, and Nicolas Papernot. [n. d.]. High Accuracy and High Fidelity Extraction of Neural Networks. In *Proc. {USENIX} Security 2020.*

[22] Jinyuan Jia and Neil Zhenqiang Gong. [n. d.]. AttriGuard: A Practical Defense Against Attribute Inference Attacks via Adversarial Machine Learning. In *Proc. USENIX Security 2018.*

[23] Mika Juuti, Sebastian Szyller, Samuel Marchal, and N. Asokan. [n. d.]. PRADA: Protecting Against DNN Model Stealing Attacks. In *Proc. EuroS&P 2019.*

[24] Manish Kesarwani, Bhaskar Mukhoty, Vijay Arya, and Sameep Mehta. [n. d.]. Model Extraction Warning in MLaaS Paradigm. In *Proc. ACSAC 2018.*

[25] Thomas N. Kipf and Max Welling. [n. d.]. Semi-Supervised Classification with Graph Convolutional Networks. In *Proc. ICLR 2017.*

[26] Johannes Klicpera, Aleksandar Bojchevski, and Stephan Günnemann. [n. d.]. Predict then Propagate: Graph Neural Networks meet Personalized PageRank. In *Proc. ICLR 2019.*

[27] Taesung Lee, Benjamin Edwards, Ian Molloy, and Dong Su. [n. d.]. Defending Against Neural Network Model Stealing Attacks Using Deceptive Perturbations. In *2019 IEEE Security and Privacy Workshops.*

[28] Jia Li, Honglei Zhang, Zhichao Han, Yu Rong, Hong Cheng, and Junzhou Huang. [n. d.]. Adversarial Attack on Community Detection by Hiding Individuals. In *Proc. WWW 2020.*

[29] Shaofeng Li, Shiqing Ma, Minhui Xue, and Benjamin Zi Hao Zhao. 2020. Deep Learning Backdoors. *CoRR* abs/2007.08273 (2020).

[30] Yujia Li, Oriol Vinyals, Chris Dyer, Razvan Pascanu, and Peter W. Battaglia. 2018. Learning Deep Generative Models of Graphs. *CoRR* abs/1803.03324 (2018).

[31] Peiyuan Liao, Han Zhao, Keyulu Xu, Tommi S. Jaakkola, Geoffrey J. Gordon, Stefanie Jegelka, and Ruslan Salakhutdinov. 2020. Graph Adversarial Networks: Protecting Information against Adversarial Attacks. *CoRR* abs/2009.13504 (2020).

[32] Jiaqi Ma, Shuangrui Ding, and Qiaozhu Mei. 2020. Towards More Practical Adversarial Attacks on Graph Neural Networks. In *Proc. NeurIPS.*

[33] Smitha Milli, Ludwig Schmidt, Anca D. Dragan, and Moritz Hardt. 2019. Model Reconstruction from Model Explanations. In *Proc. the Conference on Fairness, Accountability, and Transparency.*

[34] Xichuan Niu, Bofang Li, Chenliang Li, Rong Xiao, Haochuan Sun, Hongbo Deng, and Zhenzhong Chen. 2020. A Dual Heterogeneous Graph Attention Network to Improve Long-Tail Performance for Shop Search in E-Commerce. In *KDD.* ACM, 3405–3415.

[35] Seong Joon Oh, Bernt Schiele, and Mario Fritz. 2019. Towards Reverse-Engineering Black-Box Neural Networks. In *Explainable AI: Interpreting, Explaining and Visualizing Deep Learning.*

[36] Tribhuvanesh Orekondy, Bernt Schiele, and Mario Fritz. [n. d.]. Knockoff Nets: Stealing Functionality of Black-Box Models. In *Proc. CVPR 2019.*

[37] Soham Pal, Yash Gupta, Aditya Shukla, Aditya Kanade, Shirish K. Shevade, and Vinod Ganapathy. 2019. A framework for the extraction of Deep Neural Networks by leveraging public data. *CoRR* abs/1905.09165 (2019).

[38] Robert Nikolai Reith, Thomas Schneider, and Oleksandr Tkachenko. [n. d.]. Efficiently Stealing your Machine Learning Models. In *Proc. the 18th ACM Workshop on Privacy in the Electronic Society, WPES@CCS 2019.* ACM, 198–210.

[39] Reza Shokri, Marco Stronati, Congzheng Song, and Vitaly Shmatikov. [n. d.]. Membership Inference Attacks Against Machine Learning Models. In *Proc. SP 2017.*

[40] Julien Simon. 2019. Now Available on Amazon SageMaker: The Deep Graph Library. https://aws.amazon.com/blogs/aws/now-available-on-amazon-sagemaker-the-deep-graph-library/

[41] Florian Tramèr, Fan Zhang, Ari Juels, Michael K. Reiter, and Thomas Ristenpart. [n. d.]. Stealing Machine Learning Models via Prediction APIs. In *Proc. USENIX Security 16.*

[42] B. S. Vidyalakshmi, Raymond K. Wong, and Chi-Hung Chi. [n. d.]. User Attribute Inference in Directed Social Networks as a Service. In *Proc. SCC 2016.*

[43] Sheng Wan, Yibing Zhan, Liu Liu, Baosheng Yu, Shirui Pan, and Chen Gong. 2021. Contrastive Graph Poisson Networks: Semi-Supervised Learning with Extremely Limited Labels. In *Thirty-Fifth Conference on Neural Information Processing Systems.*

[44] Binghui Wang and Neil Zhenqiang Gong. [n. d.]. Attacking Graph-based Classification via Manipulating the Graph Structure. In *Proc. the 2019 ACM SIGSAC Conference on Computer and Communications Security, CCS 2019.* ACM, 2023–2040.

[45] Binghui Wang, Tianxiang Zhou, Minhua Lin, Pan Zhou, Ang Li, Meng Pang, Cai Fu, Hai Li, and Yiran Chen. 2020. Evasion Attacks to Graph Neural Networks via Influence Function. *CoRR* abs/2009.00203 (2020).

[46] Xiao Wang, Di Jin, Xiaochun Cao, Liang Yang, and Weixiong Zhang. 2016. Semantic Community Identification in Large Attribute Networks. In *Proc. AAAI.*

[47] Huijun Wu, Chen Wang, Yuriy Tyshetskiy, Andrew Docherty, Kai Lu, and Liming Zhu. [n. d.]. Adversarial Examples for Graph Data: Deep Insights into Attack and Defense. In *Proc. IJCAI 2019.*

[48] Jing Xu, Minhui Xue, and Stjepan Picek. [n. d.]. Explainability-based Backdoor Attacks Against Graph Neural Networks. In *Proc. WiseML@WiSec 2021: Proceedings of the 3rd ACM Workshop on Wireless Security and Machine Learning*, Christina Pöpper and Mathy Vanhoef (Eds.). ACM, 31–36.

[49] Jiaxuan You, Rex Ying, Xiang Ren, William L. Hamilton, and Jure Leskovec. 2018. GraphRNN: A Deep Generative Model for Graphs. *CoRR* abs/1802.08773 (2018).

[50] He Zhang, Bang Wu, Xiangwen Yang, Chuan Zhou, Shuo Wang, Xingliang Yuan, and Shirui Pan. 2021. Projective Ranking: A Transferable Evasion Attack Method on Graph Neural Networks. In *CIKM.* ACM, 3617–3621.

[51] Hengtong Zhang, Tianhang Zheng, Jing Gao, Chenglin Miao, Lu Su, Yaliang Li, and Kui Ren. [n. d.]. Data Poisoning Attack against Knowledge Graph Embedding. In *Proc. IJCAI 2019.*

[52] Zaixi Zhang, Jinyuan Jia, Binghui Wang, and Neil Zhenqiang Gong. 2021. Backdoor Attacks to Graph Neural Networks. In *Proc. SACMAT '21: The 26th ACM Symposium on Access Control Models and Technologies, 2021.* ACM, 15–26.

[53] Shichao Zhu, Shirui Pan, Chuan Zhou, Jia Wu, Yanan Cao, and Bin Wang. 2020. Graph Geometry Interaction Learning. In *Advances in Neural Information Processing Systems 33 (NeurIPS 2020).*

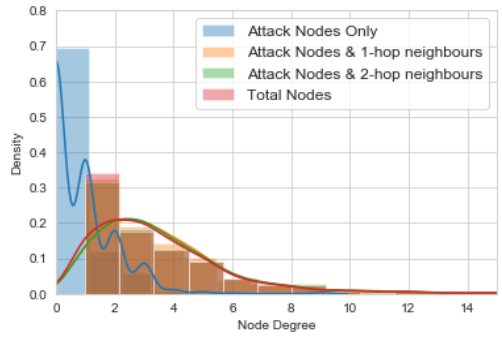
[54] Marinka Zitnik, Jure Leskovec, et al. 2018. Prioritizing network communities. *Nature communications* (2018).

[55] Daniel Zügner, Amir Akbarnejad, and Stephan Günnemann. [n. d.]. Adversarial Attacks on Neural Networks for Graph Data. In *Proc. IJCAI 2019.*

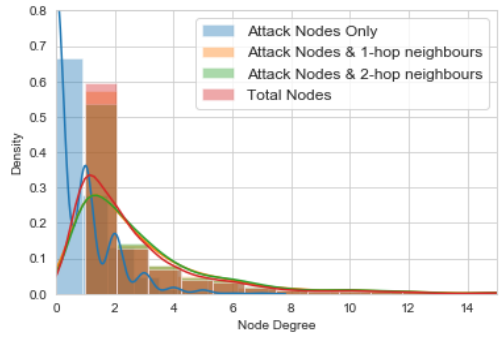
A APPENDIX

A.1 Degree distribution for Attack-0.

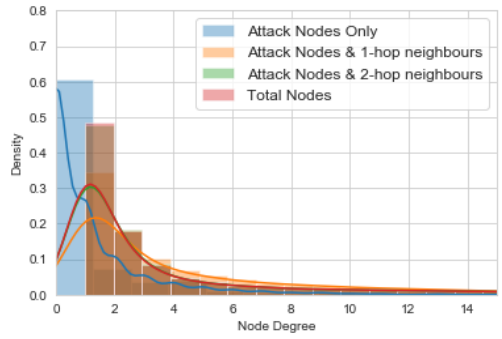
We evaluate the degree distribution when using different synthesise methods for Attack-0 as shown in Figure 10. We compare the distribution for the total nodes in the target graph, the attack nodes with their 1-hop or 2-hop neighbours, and only the attack nodes. It can be observed that, utilising the neighbours of the attack nodes can significantly synthesise the distribution of the entire graph. Therefore, it is necessary to consider the neighbours of the attack nodes and synthesise their attributes if they are inaccessible to the attackers.



(a) Cora



(b) Citeseer

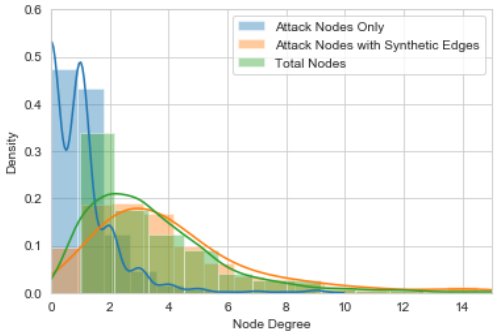


(c) Pubmed

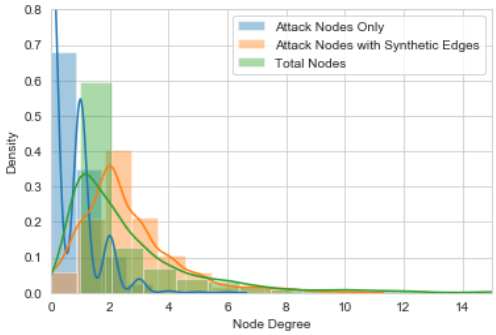
Figure 10: Degree Distribution for Attack-0

A.2 Degree distribution for Attack-1.

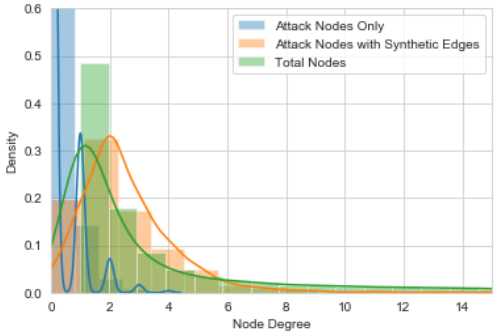
We also evaluate the degree distribution with/without synthesising methods for Attack-1 as shown in Figure 11. We compare the distribution among the total nodes in the target graph, the attack nodes with synthetic edges, and only the attack nodes. It can be found that, once applying the graph structure generation methods, the synthesised graph can be more similar to the target graph and benefits our extraction attacks.



(a) Cora



(b) Citeseer



(c) Pubmed

Figure 11: Degree Distribution for Attack-1